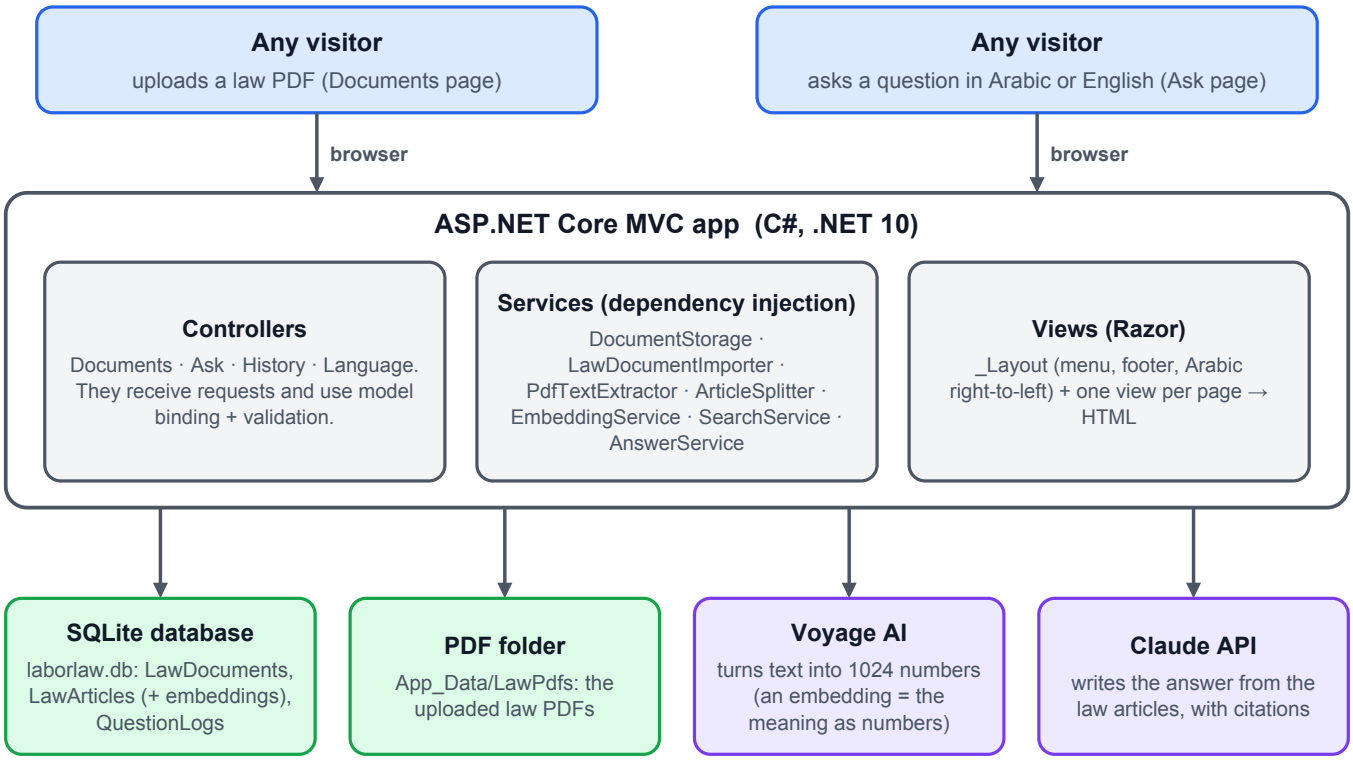
Labor Law Assistant: how the system works

An AI assistant that answers labor-law questions (Arabic + English) from the official law text, and shows the article and page it used. This document shows the parts of the system and the order in which things happen.

1. The big picture



2. RAG in three short sentences

- **R** **Retrieval:** find the 5 law articles whose meaning is closest to the question.
- **A** **Augmentation:** add those 5 articles to the request we send to Claude.
- **G** **Generation:** Claude writes the answer only from those articles and cites article + page.

Why RAG? Claude does not need to remember the law. We give it the exact official text for each question, so the answer is based on the real law and can show its source.

3. The pages of this document

Page 2	One request through ASP.NET Core MVC: how the framework works (example: uploading a PDF).
Page 3	Part A: prepare the laws. Happens once for each law PDF. Steps 3 to 5.
Page 4	Part B: answer a question. Happens for every question. Steps 6 to 8.
Page 5	Part C: going live on the internet. Steps 10 to 15.

4. The required technologies and where to find them

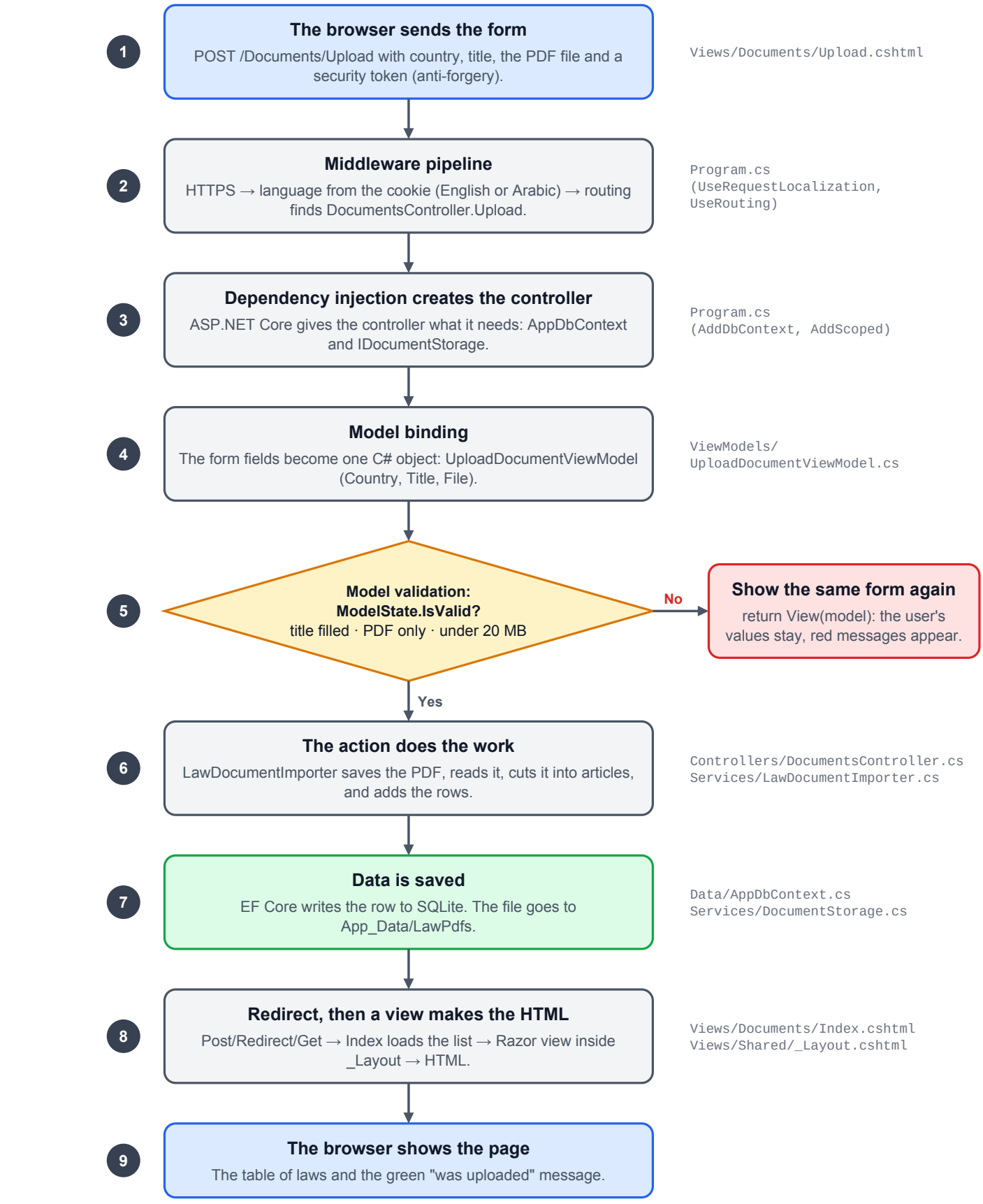
MVC	Controllers/ (logic) · Models/ + ViewModels/ (data) · Views/ (pages)
Razor views + layout	Views/Shared/_Layout.cshtml wraps every page · one .cshtml per page
Model binding	form fields → UploadDocumentViewModel in DocumentsController.Upload
Model validation	[Required] · [PdfFile] · [MaxFileSize] → ModelState.IsValid
Dependency injection	Program.cs registers AppDbContext + IDocumentStorage · constructors receive them

Person / browser · App code (C#) · Saved data · AI service (internet) · Check (yes / no) · Error path

DONE built and tested · NEXT the next step · LATER a later step

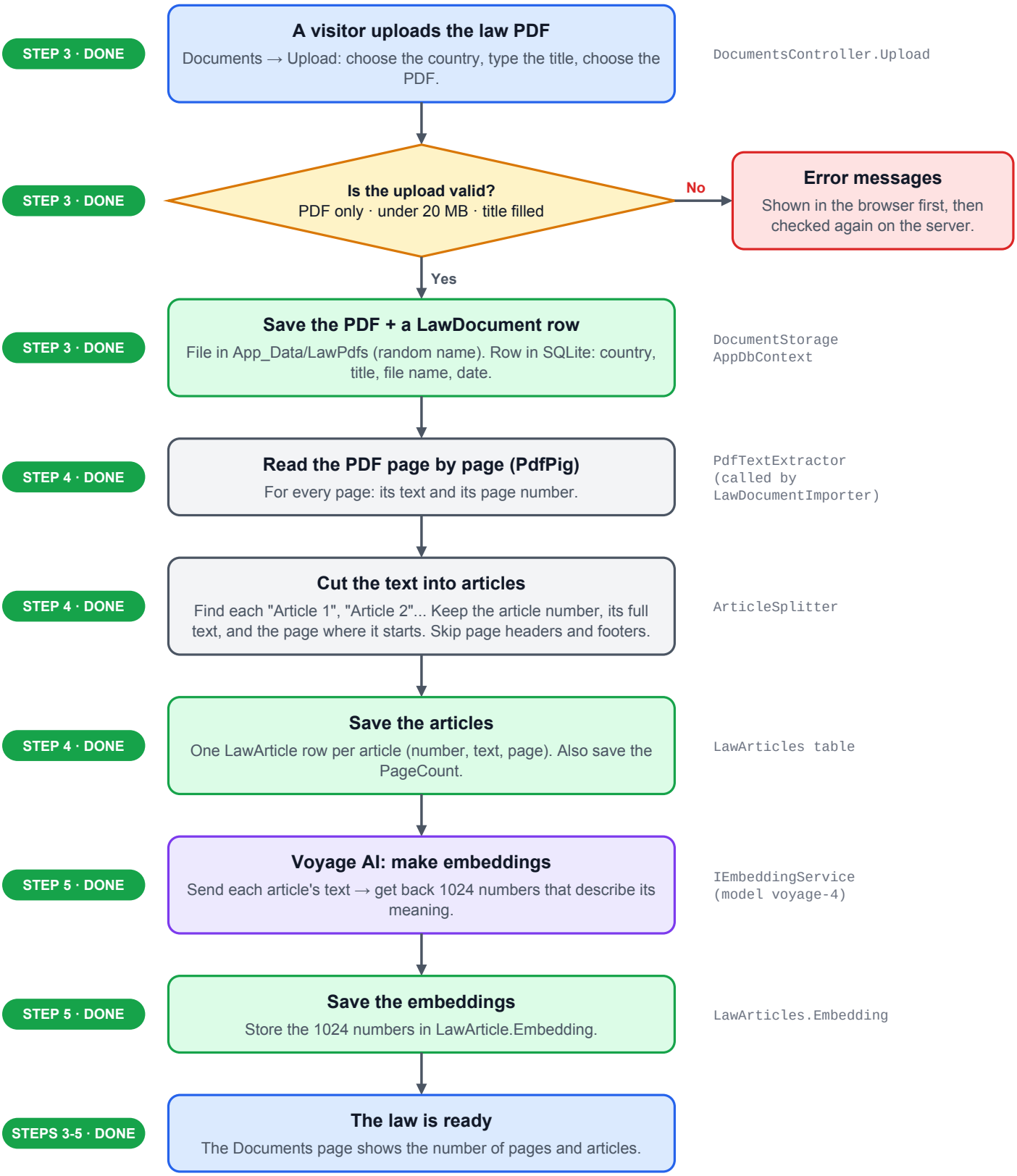
One request through ASP.NET Core MVC

Example: a visitor uploads a PDF on the Documents → Upload page. Every page of the app follows the same path. The gray text on the right is the file where it happens.



Part A: prepare the laws (once for each law PDF)

A visitor uploads the official PDF. The app cuts it into articles and stores each article with its page number and its embedding (meaning as numbers). After this, the law is ready for questions.

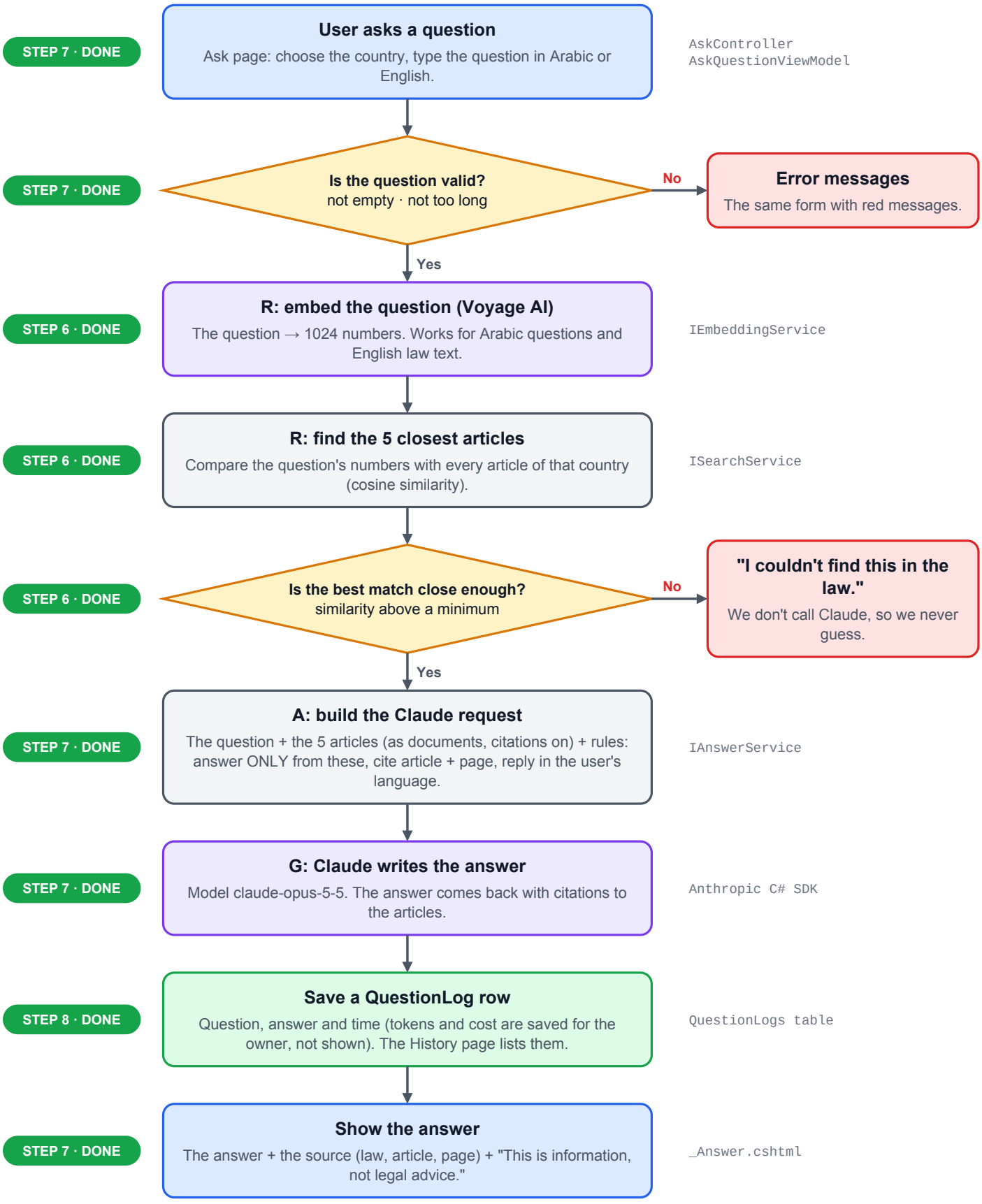


Person / browser · App code (C#) · Saved data · AI service (internet) · Check (yes / no) · Error path

DONE built and tested · NEXT the next step · LATER a later step

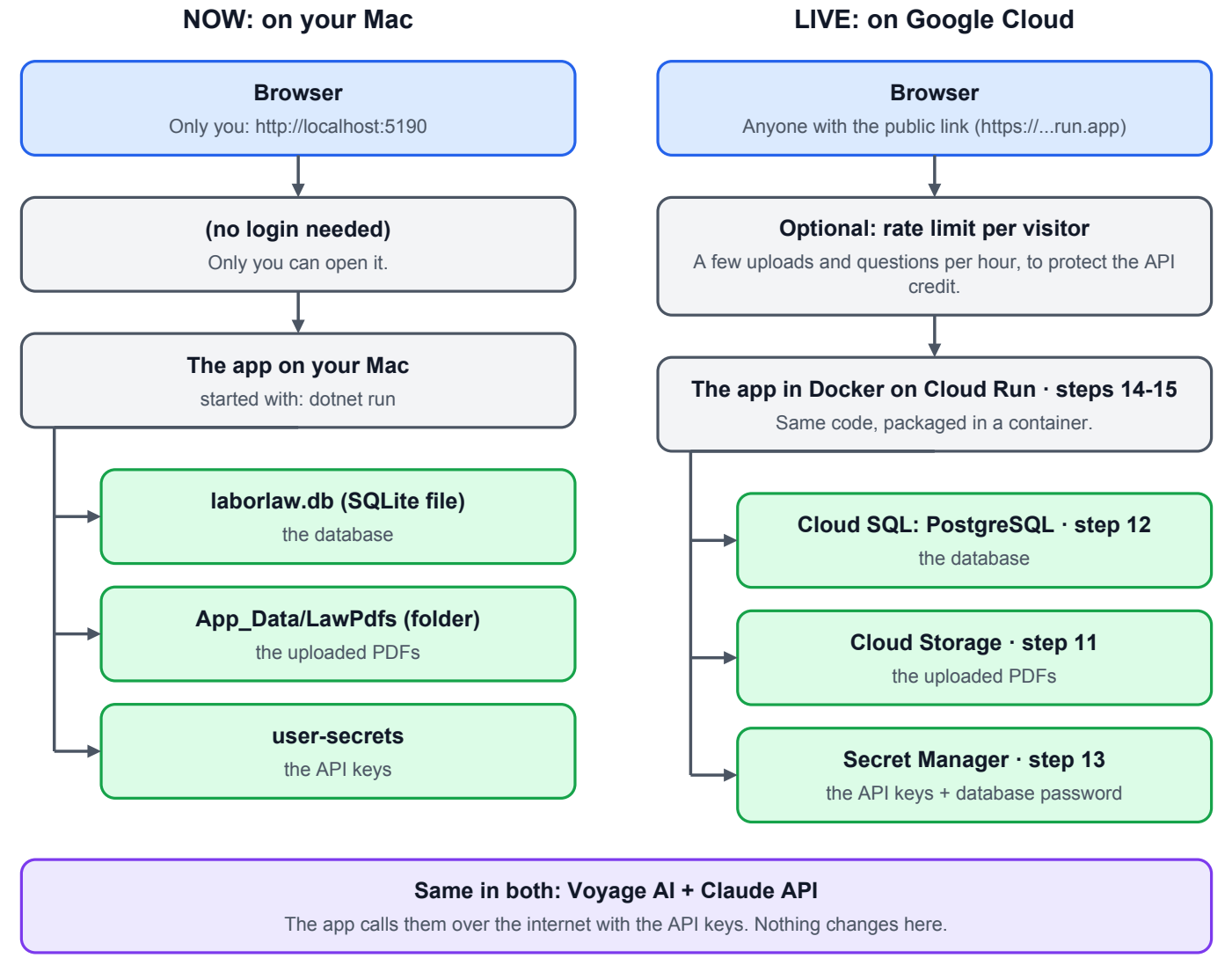
Part B: answer a question (every question)

This is the RAG part: R = find the closest articles, A = add them to the request, G = Claude writes the answer with the source.



Part C: going live (steps 10 to 15)

Today everything runs on your Mac. Live, the app runs on Google Cloud Run. A Cloud Run server is temporary (its disk is wiped on restart, and it can run several copies), so files and data move to services that keep them.



What changes in the code? Very little:

• **PDF files:** GoogleCloudDocumentStorage implements IDocumentStorage, like LocalDocumentStorage. The setting Storage:Provider chooses which one Program.cs registers. The controllers do not change: this is the benefit of dependency injection.

• **Database:** the setting Database:Provider chooses UseSqlite(...) or UseNpgsql(...). PostgresAppDbContext has its own migrations, and the tables are created automatically when the app starts. The rest of the EF Core code stays the same.

• **API keys:** Cloud Run gives the Secret Manager values to the app as environment variables (Anthropic__ApiKey...), and the app reads them like user-secrets, so no code change. Status: the code, Dockerfile and deploy script are ready (docs/DEPLOY.md); not deployed yet.

• **Simpler option:** one small server with a permanent disk (Azure App Service or a VM). Then SQLite and the App_Data folder work as they are, and steps 11 to 13 are not needed.

Person / browser | App code (C#) | Saved data | AI service (internet) | Check (yes / no) | Error path